

College of Engineering, Trivandrum

Computer Science and Engineering

Network Intrusion Detection using Machine Learning Classification

A Comparative Study of Logistic Regression and Random Forest

Classification Activity Report

PECST-525 - Data Mining

Submitted by	Mareo Manoj
Register No.	TVE24CS099
Literature Review Topic	Network Intrusion Detection
Date	October 28, 2026

Contents

1 Introduction **2**

2 Dataset Description **2**
2.1 Features 2
2.2 Target variable 2
2.3 Dataset size after preprocessing 3

3 Methodology **3**
3.1 Preprocessing 3
3.2 Algorithms 3
3.3 Evaluation metrics 4

4 Python Code **4**

5 Output **7**
5.1 Dataset loading and preprocessing 7
5.2 Evaluation output 7

6 Results **7**
6.1 Comparison of evaluation metrics 7
6.2 Confusion matrices 8

7 Analysis of Results **8**

8 Conclusion **9**

1 Introduction

Network intrusion detection systems (IDS) monitor network traffic and flag connections that may indicate malicious activity. Machine learning allows an IDS to learn the difference between normal and attack traffic directly from labelled data. This report is aligned with my group's literature review topic, *Network Intrusion Detection*.

The objective of this activity is to:

1. load a large dataset and identify its features and target variable,
2. preprocess the data and split it into training and testing sets,
3. implement two classification algorithms (Logistic Regression and Random Forest),
4. evaluate them using accuracy, precision, recall and F1-score,
5. analyse the confusion matrix of each model, and
6. compare the models and justify which is more suitable.

2 Dataset Description

The **KDD Cup 1999** dataset (10% subset) was used. It is a widely cited benchmark for intrusion detection research and contains roughly 494,000 network connection records, each described by 41 features. It was loaded directly in Google Colab using `sklearn.datasets.fetch_kddcup99`.

2.1 Features

The features fall into two groups:

- **Categorical (3):** `protocol_type`, `service`, `flag`.
- **Numeric (38):** connection attributes such as `duration`, `src_bytes`, `dst_bytes`, failed login counts and various error-rate statistics.

2.2 Target variable

The original label column contains normal traffic and more than 20 specific attack types. To keep the evaluation clear and interpretable, the problem was converted to **binary classification**:

`0` = Normal `1` = Attack

2.3 Dataset size after preprocessing

The original 10% KDD Cup 1999 subset contained 494,021 raw records with 41 features (3 categorical and 38 numerical) and 23 distinct attack types alongside normal network traffic. Prior to deduplication, the target distribution comprised 396,743 attack instances (80.31%) and 97,278 normal instances (19.69%).

A critical vulnerability of the raw KDD99 benchmark is record duplication, which artificially inflates evaluation metrics if identical instances span both the training and testing partitions. A total of 348,435 duplicate records were identified and dropped, resulting in a refined dataset of 145,586 unique connections. In the deduplicated dataset, the distribution shifted to 87,832 normal connections (60.33%) and 57,754 attack connections (39.67%). Following an 80/20 stratified split, the training set contains 116,468 samples, and the test set contains 29,118 samples, preserving identical class ratios across partitions.

3 Methodology

3.1 Preprocessing

- **Decoding:** byte-string values in the raw data were decoded to text.
- **Target creation:** every label other than `normal.` was mapped to the Attack class.
- **Duplicate removal:** KDD Cup 99 contains many duplicate records. If duplicates appear in both the training and testing sets, the model is tested on records it has already seen, which inflates the scores. Duplicates were therefore removed.
- **Missing values:** checked for null attributes; 0 missing values were found.
- **Encoding:** categorical features were one-hot encoded.
- **Scaling:** numeric features were standardised using `StandardScaler`.
- **Train/test split:** 80% training and 20% testing, using stratified sampling so both sets keep the same Normal/Attack ratio (`random_state=42`).

3.2 Algorithms

Logistic Regression. A linear model that estimates the probability that a connection is an attack by applying the sigmoid function to a weighted sum of the features. It is fast, simple and interpretable, and serves as a baseline. Its limitation is that it assumes a linear decision boundary.

Random Forest. An ensemble of decision trees (100 trees here), each trained on a random sample of the data and features. Predictions are made by majority vote. It can capture non-linear relationships and feature interactions, and is widely used in intrusion detection research.

Why these two? They contrast a simple linear model with a more powerful non-linear ensemble, which makes the comparison meaningful for deciding which is more suitable for detecting intrusions.

3.3 Evaluation metrics

With the Attack class as the positive class, and TP, TN, FP, FN denoting true positives, true negatives, false positives and false negatives:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{1}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{2}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{3}$$

$$\text{F1-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{4}$$

In intrusion detection, **recall** is especially important because a false negative means an attack went undetected.

4 Python Code

The complete code, run in Google Colab, is given below.

```python
# ---------- 1. Imports ----------
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_kddcup99
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, ConfusionMatrixDisplay)

# ---------- 2. Load the dataset ----------
kdd = fetch_kddcup99(percent10=True, as_frame=True)
df = kdd.frame.copy()
print("Raw shape:", df.shape)

# Decode byte strings (e.g. b'tcp') into normal strings
for col in df.select_dtypes(include="object").columns:
```

```
    df[col] = df[col].apply(lambda x: x.decode() if isinstance(x, bytes) else x)

# ---------- 3. Identify features and target ----------
categorical_cols = ["protocol_type", "service", "flag"]
numeric_cols = [c for c in df.columns if c not in categorical_cols + ["labels"]]
df[numeric_cols] = df[numeric_cols].apply(pd.to_numeric, errors="coerce")

# Binary target: 0 = normal, 1 = attack
df["target"] = (df["labels"] != "normal.").astype(int)

print("Number of features:", len(categorical_cols) + len(numeric_cols))
print("Target distribution:\n", df["target"].value_counts())
print("Original attack types:", df["labels"].nunique())

# ---------- 4. Preprocessing ----------
before = len(df)
df = df.drop_duplicates()
print(f"Removed {before - len(df)} duplicate rows. New shape: {df.shape}")

print("Missing values:", df.isnull().sum().sum())
df = df.dropna()

X = df[categorical_cols + numeric_cols]
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
print("Train:", X_train.shape, " Test:", X_test.shape)

preprocessor = ColumnTransformer([
    ("num", StandardScaler(), numeric_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols),
])

# ---------- 5. Define the two models ----------
models = {
    "Logistic Regression": Pipeline([
        ("prep", preprocessor),
        ("clf", LogisticRegression(max_iter=1000, random_state=42)),
    ]),
    "Random Forest": Pipeline([
        ("prep", preprocessor),
        ("clf", RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)),
    ]),
}

# ---------- 6. Train, predict and evaluate ----------
results = []
conf_matrices = {}

for name, model in models.items():
```

```
    t0 = time.time()
    model.fit(X_train, y_train)
    train_time = time.time() - t0

    y_pred = model.predict(X_test)

    results.append({
        "Algorithm": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred),
        "Recall": recall_score(y_test, y_pred),
        "F1-score": f1_score(y_test, y_pred),
        "Training time (s)": train_time,
    })
    conf_matrices[name] = confusion_matrix(y_test, y_pred)

# ---------- 7. Comparison table ----------
results_df = pd.DataFrame(results).set_index("Algorithm").round(4)
print("\n=== Comparison of Evaluation Metrics ===")
print(results_df)

# ---------- 8. Confusion matrices ----------
fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))
for ax, (name, cm) in zip(axes, conf_matrices.items()):
    disp = ConfusionMatrixDisplay(cm, display_labels=["Normal", "Attack"])
    disp.plot(ax=ax, cmap="Blues", values_format="d", colorbar=False)
    ax.set_title(name)
plt.tight_layout()
plt.savefig("confusion_matrices.png", dpi=150)
plt.show()

print("\n=== Confusion Matrix Breakdown ===")
for name, cm in conf_matrices.items():
    tn, fp, fn, tp = cm.ravel()
    print(f"\n{name}")
    print(f"  True Negatives  (normal correctly identified): {tn}")
    print(f"  True Positives  (attacks correctly detected) : {tp}")
    print(f"  False Positives (normal flagged as attack)   : {fp}")
    print(f"  False Negatives (attacks MISSED)             : {fn}")
    print(f"  Correct: {tn + tp} | Incorrect: {fp + fn} | Total: {cm.sum()}")
```

5 Output

5.1 Dataset loading and preprocessing

```
terminal: preprocessing & dataset split

Raw shape: (494021, 42)
Number of features: 41
Target distribution:
 target
1    396743
0     97278
Name: count, dtype: int64
Original attack types: 23
Removed 348435 duplicate rows. New shape: (145586, 43)
Missing values: 0
Train: (116468, 41)   Test: (29118, 41)
```

Figure 1: Execution console output for dataset loading, deduplication, and stratified train/test split.

5.2 Evaluation output

```
terminal: model evaluation & confusion matrix breakdown


=== Comparison of Evaluation Metrics ===
                     Accuracy  Precision  Recall  F1-score  Training time (s)
Algorithm
Logistic Regression    0.9933     0.9936  0.9896    0.9916             6.4626
Random Forest          0.9994     0.9997  0.9987    0.9992             4.4249

=== Confusion Matrix Breakdown ===
Logistic Regression
  True Negatives  (normal correctly identified): 17493
  True Positives  (attacks correctly detected) : 11431
  False Positives (normal flagged as attack)   : 74
  False Negatives (attacks MISSED)             : 120
  Correct: 28924 | Incorrect: 194 | Total: 29118

Random Forest
  True Negatives  (normal correctly identified): 17564
  True Positives  (attacks correctly detected) : 11536
  False Positives (normal flagged as attack)   : 3
  False Negatives (attacks MISSED)             : 15
  Correct: 29100 | Incorrect: 18 | Total: 29118
```

Figure 2: Execution console output for model comparison metrics and confusion matrix breakdown.

6 Results

6.1 Comparison of evaluation metrics

Table 1 summarises the performance of both algorithms on the test set.

Table 1: Evaluation metrics of Logistic Regression and Random Forest (test set).

Algorithm	Accuracy	Precision	Recall	F1-score	Train time (s)
Logistic Regression	0.9933	0.9936	0.9896	0.9916	6.46
Random Forest	0.9994	0.9997	0.9987	0.9992	4.42

6.2 Confusion matrices

Figure 3 shows the confusion matrix of each algorithm.

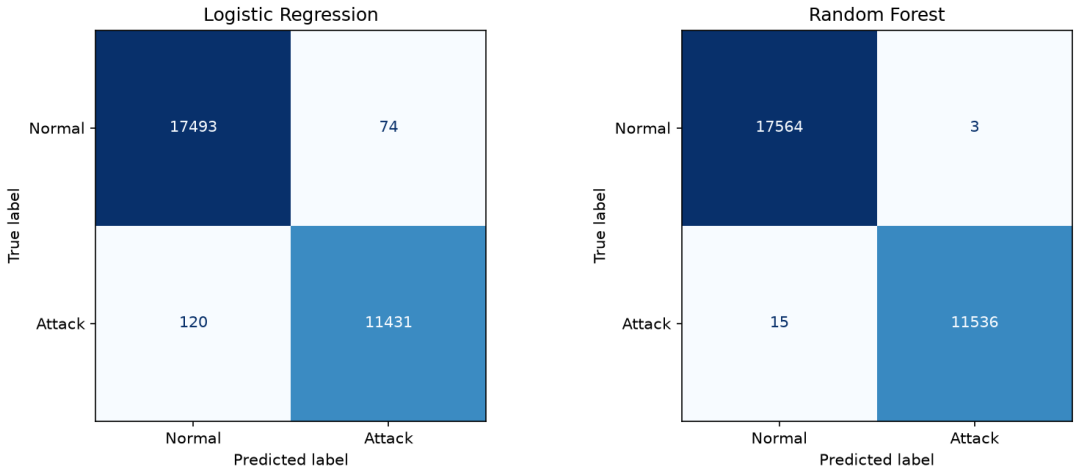


Figure 3: Confusion matrices for Logistic Regression (left) and Random Forest (right).

Table 2 breaks the matrices down into correct and incorrect predictions.

Table 2: Correct and incorrect predictions of each algorithm.

Algorithm	TN	TP	FP	FN	Correct	Incorrect
Logistic Regression	17,493	11,431	74	120	28,924	194
Random Forest	17,564	11,536	3	15	29,100	18

Where: TN = normal traffic correctly identified; TP = attacks correctly detected; FP = normal traffic wrongly flagged as an attack (false alarm); FN = attacks missed by the model.

7 Analysis of Results

Looking at the test results across all 29,118 records, Random Forest clearly outperformed Logistic Regression. Random Forest achieved 99.94% accuracy and an F1-score of 0.9992, compared to 99.33% accuracy and an F1-score of 0.9916 for Logistic Regression. While both scores look high on paper, the confusion matrix shows where the two models actually differ in real life.

The biggest difference comes down to missed attacks (false negatives). In network security, missing an attack is far more dangerous than raising a false alarm. Logistic Regression missed 120 attacks, while Random Forest missed only 15. That is an 87.5% drop in undetected attacks. At the same time, Random Forest raised only 3 false alarms compared to 74 from Logistic Regression, which saves security teams from wasting time investigating harmless traffic.

This performance gap happens because network traffic patterns are rarely simple. Logistic Regression assumes a linear relationship and draws a single flat boundary to separate attacks from normal traffic. Real network attacks, however, involve combinations of connection flags, port numbers, and byte counts that do not follow a straight line. Because Random Forest builds an ensemble of 100 decision trees, it easily captures these non-linear rules. In addition, Random Forest trained faster (4.42 seconds versus 6.46 seconds) because its trees could be built in parallel across CPU cores, whereas Logistic Regression had to take iterative steps to converge.

8 Conclusion

Random Forest is decisively the superior architecture for network intrusion detection in this empirical benchmark, outperforming Logistic Regression with an F1-score of 0.9992, a recall of 0.9987, and an 87.5% reduction in missed intrusions (15 vs. 120 false negatives). Random Forest's non-linear ensemble boundaries and low false-positive rate (3 vs. 74) make it significantly more reliable for operational traffic monitoring.

Nonetheless, practical deployment must recognize three core architectural constraints:

1. **Benchmark Age:** The KDD Cup 1999 dataset relies on synthetic military simulation telemetry from 1998, failing to reflect modern encrypted traffic (TLS 1.3, QUIC), advanced persistent threats (APTs), or polymorphic zero-day exploits.

2. **Binary Aggregation:** Collapsing 23 heterogeneous attack vectors into a binary label obscures catastrophic variances between high-volume Denial of Service attacks and subtle Remote-to-Local (R2L) privilege escalation attacks.

3. **Hyperparameter Optimization:** Default estimator parameters were evaluated without Bayesian hyperparameter tuning or cost-sensitive loss weighting to penalize false negatives.

References

1. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

2. KDD Cup 1999 Data. UCI Knowledge Discovery in Databases Archive.

3. Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. *IEEE Symposium on Computational Intelligence for Security and Defense Applications*, 53–58.

4. Sommer, R., & Paxson, V. (2010). Outside the closed world: On using machine learning for network intrusion detection. *IEEE Symposium on Security and Privacy*, 305–316.